Topic 25 - Operational Data Management with Google Sheets & Tabular Stores

Step 1: Create the In-n8n Mock Google Sheets Server (Zero-Blocker Setup)

1. Create a new workflow named Mock Google Sheets Server.

2. Add a **Webhook Node** named Mock_Sheets_Inventory_GET:

HTTP Method: GET

Path: mock/sheets/inventory

Response Mode: Using 'Respond to Webhook' Node

3. Connect a **Code Node** returning current inventory rows:

```
const staticData = $getWorkflowStaticData('global');
// Initialize warehouse inventory if not yet set
if (!staticData.inventory) {
 staticData.inventory = [
 { sku: "SKU-KB-01", itemName: "Wireless Mechanical Keyboard", stockOnHand: 25, reorderPoint: 5, unitPrice: 89.99 },  { sku: "SKU-MO-02", itemName: "Ergonomic Optical Mouse", stockOnHand: 6, reorderPoint: 5, unitPrice: 49.99 },  { sku: "SKU-DS-03", itemName: "Thunderbolt 4 Docking Station", stockOnHand: 1, reorderPoint: 2, unitPrice: 199.99 }  ];
}
return { json: { rows: staticData.inventory } };
```

4. Connect to a **Respond to Webhook Node** (Response Body: ={{ $json.rows }}). **5.** Add a **Webhook Node** named Mock_Sheets_Inventory_PUT:

HTTP Method: PUT

Path: mock/sheets/inventory

Response Mode: Using 'Respond to Webhook' Node

6. Connect a **Code Node** updating the matching SKU:

```
const staticData = $getWorkflowStaticData('global');
const update = $input.first().json.body || {};
const sku = update.sku;
const newStock = update.newStock;

if (staticData.inventory) {
 const idx = staticData.inventory.findIndex(r => r.sku === sku);
 if (idx !== -1) {
 staticData.inventory[idx].stockOnHand = newStock;
 }
}
return { json: { status: "ROW_UPDATED", sku: sku, stockOnHand: newStock } }
```

7. Connect to a Respond node returning the update confirmation.

8. Add a **Webhook Node** named Mock_Sheets_Orders_POST:

HTTP Method: POST

Path: mock/sheets/orders

Response Mode: Using 'Respond to Webhook' Node

9. Connect a Respond node: Code 201, body {"status": "ROW_APPENDED", "orderId": $json.body.orderId}.
10. Save and activate the mock server workflow.

Step 2: Configure Main Workflow Trigger & Simulated Orders

1. **Create Workflow & Manual Trigger:**

   In your n8n dashboard, click **+ Add workflow** in the top-right corner.

   Rename to: Topic 10 - Operational Data Management with Google Sheets & Tabular Stores.

   Click **+ Add first step...**, search for **Manual Trigger**, and place it.
   Rename to: Manual_Trigger.

2. **Add Simulated Orders Node:**

   Hover over the right connector handle of Manual_Trigger, click **+**, search for **Code**, and add the node.

   Rename to: Code_SimulateOrders.

3. **Configure Parameters (Parameters Tab):**

   Language: Select **JavaScript**.

   Mode: Select **Run Once for All Items**.

   JavaScript Code: Paste the 3 order scenarios (ample stock, low-stock reorder trigger, out-of-stock):

```
return [
 {
 json: {
 orderId: "ORD-7001",
 customerEmail: "david@enterprise.com",
 sku: "SKU-KB-01",
 quantity: 2,
 notes: "Standard order - ample stock available"
 }
 },
 {
 json: {
 orderId: "ORD-7002",
 customerEmail: "elena@startup.io",
 sku: "SKU-MO-02",
 quantity: 2,
 notes: "Will breach reorder point (6 - 2 = 4 <= 5)"
 }
 },
 {
 json: {
 orderId: "ORD-7003",
 customerEmail: "frank@consulting.org",
```

```
sku: "SKU-DS-03",
quantity: 5,
notes: "Stockout scenario (requested 5, only 1 on hand)"
}
}
];
```

4. **Test Node Checkpoint:**

   Click **Test step**. Verify 3 simulated order items appear in Output.

Step 3: Fetch Current Inventory Rows (Native Google Sheets Node or Zero-Blocker HTTP Mock)

Youcan configure this step using either n8n's **Native Google Sheets Node** (if you have connected Google OAuth credentials) or the **Zero-Blocker HTTP Mock** (zero setup required):

Approach A: Native Google Sheets Node (`GoogleSheets_ReadInventory`)

1. From `Code_SimulateOrders`, click **+**, search for **Google Sheets**, and select the **Google Sheets** node (`n8n-nodes-base.googleSheets`).
2. Rename to: `GoogleSheets_ReadInventory`.
3. **Parameters Tab:**

   Credential to connect with: Select your configured Google Sheets OAuth2 / Service Account credential.

   Resource: Select **Document** (or `Sheet within Document`).

   Operation: Select **Get Many** (or `Read Rows`).

   Document: Select **By URL** or **From List** $ ightarrow$ choose your `Enterprise_Inventory` spreadsheet.

   Sheet: Select `Inventory`.

   Return All: Toggleto **ON**.

Approach B: Zero-Blocker HTTP Mock (`HTTP_GetInventoryRows`)

1. From `Code_SimulateOrders`, click **+**, search for **HTTP Request**, and rename to: `HTTP_GetInventoryRows`.
2. **Parameters Tab:**

   Method: Select **GET**.

   URL: Toggleto **Expression** (**fx**), enter:

   DummyJSON Option A: `https://dummyjson.com/c/<your-id>`

   Postman Option B: `https://{{mock-id}}.mock.pstmn.io/sheets/inventory` Local

   Webhook Option C: `http://localhost:5678/webhook/mock/sheets/inventory`

Authentication: Select **None**.

3. **Test Node Checkpoint:**

   Click **Test step**. Verify current inventory rows (with `sku`, `stockOnHand`, `reorderPoint`, `unitPrice`) are returned.

Step 4: Reconcile Stock & Compute Balances (`Code_ReconcileStock`)

1. **Add Node to Canvas:**

   From `HTTP_GetInventoryRows` (or `GoogleSheets_ReadInventory`), click **+**, search for **Code**, and add the node.

   Rename to: `Code_ReconcileStock`.

2. **Configure Parameters (Parameters Tab):**

   Language: Select **JavaScript**.

   Mode: Select **Run Once for Each Item**.

   JavaScript Code: Paste the stock reconciliation engine:

```
const order = $node['Code_SimulateOrders'].json;
const inventoryRows = $input.item.json;

// Find matching row by SKU
                        const matchedItem = (Array.isArray(inventoryRows) ? inventoryRows : [inventoryRows]).find(r => r.sku === order.sku);

if (!matchedItem) {
 return {
 json: {
 orderId: order.orderId,
 sku: order.sku,
 isAvailable: false,
 reason: "SKU_NOT_FOUND_IN_INVENTORY"
 }
 };
 }

const currentStock = Number(matchedItem.stockOnHand || 0);
const orderQty = Number(order.quantity || 0);
const reorderPoint = Number(matchedItem.reorderPoint || 0);
const unitPrice = Number(matchedItem.unitPrice || 0);

const isAvailable = currentStock >= orderQty;
const newStock = isAvailable ? currentStock - orderQty : currentStock;
const needsReorder = isAvailable && (newStock <= reorderPoint);
const totalPrice = Number((orderQty * unitPrice).toFixed(2));

return {
 json: {
 orderId: order.orderId,
 customerEmail: order.customerEmail,
 sku: order.sku,
 itemName: matchedItem.itemName,
 quantityOrdered: orderQty,
 unitPrice: unitPrice,
 totalPrice: totalPrice,
 previousStock: currentStock,
 newStock: newStock,
 reorderPoint: reorderPoint,
 isAvailable: isAvailable,
 needsReorder: needsReorder,
 timestamp: new Date().toISOString()
 }
 };
```

3. **Test Node Checkpoint:**

   Click **Test step**. Verify that ORD-7001 and ORD-7002 evaluate `isAvailable: true`, while ORD-7003 evaluates `isAvailable: false`.

Step 5: Evaluate Stock Availability (IF_HasSufficientStock)

1. **Add Node to Canvas:**
   From Code_ReconcileStock, click **+**, search for **If**, and select the **If** node.
   Rename to: IF_HasSufficientStock.
2. **Configure Parameters (Parameters Tab):**
   Click **Add Condition**.
   Value 1: Switch toggle to **Expression (fx)** and enter:

```
={{ $json.isAvailable }}
```

   Operation: Select **Boolean** $ ightarrow$ **Equal**.
   Value 2: Switch to **Expression (fx)** and enter {{ true }}.
3. **Connect False Branch (Out-of-Stock Quarantine):**
   From the **false (bottom) output handle**, add a **Code** node named Log_StockoutRejection. Set **Mode** to Run Once for Each Item, and paste:

```
return {
 json: {
 event: "ORDER_REJECTED_STOCKOUT",
 orderId: $json.orderId,
 sku: $json.sku,
 requestedQuantity: $json.quantityOrdered,
 availableStock: $json.previousStock,
 rejectedAt: new Date().toISOString()
 }
};
```

   Click **Test step** to verify.

Step 6: Update Inventory Row (Native Google Sheets or Zero-Blocker Mock)

Approach A: Native Google Sheets Node (GoogleSheets_UpdateStock)

Hover over **true (top) output handle** of IF_HasSufficientStock, add **Google Sheets** node. **Operation:** Update Row.
Document: Enterprise_Inventory $ ightarrow$ Sheet: Inventory.
Match Column: sku = ={{ $json.sku }}.
Update Fields: stockOnHand = ={{ $json.newStock }}.

Approach B: Zero-Blocker HTTP Mock (HTTP_UpdateInventoryRow)

1. From the **true (top) output handle** of IF_HasSufficientStock, add an **HTTP Request** node named HTTP_UpdateInventoryRow.
2. **Parameters Tab:**

Method: Select **PATCH** (or POST).

URL: Toggleto **Expression (fx)**, enter: https://dummyjson.com/c/<your-id> (or mock server URL).

Send Body: Switch toggle to **ON**.

Body Content Type: Select **JSON**.

Specify Body: Select **Using JSON**.

JSON: Switch to **Expression (fx)** and enter:

```
{{ { "sku": $json.sku, "updatedStock": $json.newStock, "updatedAt": $now.toISO() } }}
```

3. **Test Node Checkpoint:**

   Click **Test step**. Verify row update confirmation.

Step 7: Append Order to Ledger Sheet (Native Google Sheets or Zero-Blocker Mock)

Approach A: Native Google Sheets Node (GoogleSheets_AppendOrder)

Connect from HTTP_UpdateInventoryRow (or GoogleSheets_UpdateStock), add **Google Sheets** node.

Operation: Append Row.

Document: Enterprise_Inventory $ ightarrow$ Sheet: Orders_Ledger.

Columns to Append: Map orderId, customerEmail, sku, quantityOrdered, totalPrice, timestamp.

Approach B: Zero-Blocker HTTP Mock (HTTP_AppendOrderLedger)

1. From HTTP_UpdateInventoryRow, click **+**, search for **HTTP Request**, and rename to: HTTP_AppendOrderLedger.
2. **Parameters Tab:**

   Method: Select **POST**.

   URL: Toggleto **Expression (fx)**, enter mock orders ledger endpoint.

   Send Body: Switch toggle to **ON**.

   Body Content Type: Select **JSON**.

   Specify Body: Select **Using JSON**.

   JSON: Switch to **Expression (fx)** and enter:

```
{{
{
"orderId": $json.orderId,
"customerEmail": $json.customerEmail,
"sku": $json.sku,
"quantity": $json.quantityOrdered,
"totalPrice": $json.totalPrice,
"status": "FULFILLED",
```

```
"timestamp": new Date().toISOString()
}
}}
```

3. **Test Node Checkpoint:**

   Click **Test step**. Verify HTTP status 201 Created.

Step 8: Low-Stock Threshold Branching (IF_IsLowStockTrigger)

1. **Add Node to Canvas:**

   From HTTP_AppendOrderLedger, click **+**, search for **If**, and select the **If** node. Rename to: IF_IsLowStockTrigger.

2. **Configure Parameters (Parameters Tab):**

   Click **Add Condition**.

   Value 1: Switch toggle to **Expression (fx)** and enter:

   ```
   ={{ $json.needsReorder }}
   ```

   Operation: Select **Boolean** $ ightarrow$ **Equal**.

   Value 2: Switch to **Expression (fx)** and enter {{ true }}.

3. **True Branch (Procurement Reorder Alert):**

   Hover over the **true (top) output handle**, click **+**, search for **Code**, and rename to: Code_BuildReorderAlert.

   Set **Mode** to Run Once for Each Item, and paste:

   ```
   return {
   json: {
   alertType: "PROCUREMENT_LOW_STOCK_WARNING",
   sku: $json.sku,
   itemName: $json.itemName,
   remainingStock: $json.newStock,
   reorderPoint: $json.reorderPoint,
   suggestedReorderQuantity: 25,
   priority: "HIGH",
   timestamp: new Date().toISOString()
   }
   };
   ```

   Click **Test step** to verify.

Step 9: Build Fulfillment Audit Summary (Code_AuditSummary)

1. **Add Node & Converge Branches:**

   Connect both Code_BuildReorderAlert and the **false (bottom) output handle** of IF_IsLowStockTrigger into a **Code** node named

Code_AuditSummary.

2. **Configure Parameters (Parameters Tab):**

Language: Select **JavaScript**.

Mode: Select **Run Once for All Items**.

JavaScript Code: Paste:

```
const items = $input.all();
return [{
 json: {
 batchStatus: "INVENTORY_SYNC_COMPLETED",
 totalOrdersProcessed: items.length,
 synchronizedAt: new Date().toISOString()
 }
}];
```

3. **Test Node Checkpoint:**

Click **Test step**. Verify the consolidated audit card.